

Relatório Técnico: Implementação de Métodos Numéricos

Análise de Convergência, Erro e Desempenho Computacional

Guilherme Froner

Ciência da Computação - UFSM

Setembro de 2026

1 Introdução

Este relatório apresenta a implementação, em C++, de métodos numéricos para a determinação de raízes de funções. O projeto utiliza abstrações numéricas, incluindo números duais para diferenciação automática, além de mecanismos de validação de domínio e tratamento de valores não finitos, como *NaN* e infinito, a fim de tornar as execuções mais seguras e confiáveis.

2 Formulação Matemática

2.1 Números Duais

2.1.1 Definição e Motivação

A diferenciação numérica tradicional por diferenças finitas sofre de erros de truncamento e de arredondamento de ponto flutuante, os quais se propagam em cálculos iterativos. Por outro lado, a diferenciação simbólica gera expressões excessivamente complexas e não lida nativamente com estruturas de controle de fluxo de programação, como laços e condicionais. A Diferenciação Automática em Modo *Forward* resolve essas limitações utilizando a álgebra dos **Números Duais**.

Decidi utilizar esta abordagem porque, estudando *machine learning*, me deparei com algoritmos semelhantes aplicadas em implementações no *github* e explicações no *Youtube*. Assim, não dependendo necessariamente de bibliotecas externas para calcular a derivada.

Um número dual z é uma extensão dos números reais definida na forma:

$$z = a + b\epsilon, \quad a, b \in \mathbb{R} \tag{1}$$

onde a representa a componente real (ou valor primal da função) e b representa a componente dual (ou derivada/tangente). O elemento ϵ é uma unidade infinitesimal não nula ($\epsilon \neq 0$), porém **nilpotente** de ordem 2:

$$\epsilon^2 = 0 \quad (2)$$

Matematicamente, ϵ pode ser representado pela matriz nilpotente $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$, cuja propriedade garante que $\epsilon^2 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$.

2.1.2 Dedução a partir da Série de Taylor

A justificativa matemática para o uso de números duais no cálculo exato de derivadas provém da expansão em Série de Taylor de uma função infinitamente diferenciável $f(x)$ ao redor de um ponto a :

$$f(x) = f(a) + f'(a)(x - a) + \sum_{k=2}^{\infty} \frac{f^{(k)}(a)}{k!} (x - a)^k \quad (3)$$

Ao avaliar a função substituindo a variável x pelo número dual $x = a + b\epsilon$, o termo $(x - a)$ torna-se simplesmente $b\epsilon$:

$$f(a + b\epsilon) = f(a) + f'(a)(b\epsilon) + \sum_{k=2}^{\infty} \frac{f^{(k)}(a)}{k!} b^k \epsilon^k \quad (4)$$

Como $\epsilon^k = 0$ para todo $k \geq 2$, todos os termos de ordem superior da série anulam-se exatamente:

$$f(a + b\epsilon) = f(a) + f'(a)b\epsilon + 0 = f(a) + f'(a)b\epsilon \quad (5)$$

Ao definir a semente da derivada inicial como $b = 1$ (ou seja, $\frac{dx}{dx} = 1$), obtém-se a relação fundamental da Diferenciação Automática:

$$f(a + \epsilon) = f(a) + f'(a)\epsilon \quad (6)$$

Esta identidade demonstra que avaliar uma função f sobre um número dual $a + \epsilon$ resulta em um número dual cuja parte real é o valor da função $f(a)$ e cuja parte dual é a derivada exata $f'(a)$, sem qualquer erro de truncamento.

2.1.3 Propriedades Algébricas e Visualização da Regra da Cadeia

As operações aritméticas fundamentais entre números duais $z_1 = a + b\epsilon$ e $z_2 = c + d\epsilon$ derivam diretamente de suas propriedades algébricas:

- **Adição:**

$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon \quad (7)$$

O que reflete automaticamente a regra da soma de derivadas: $(f + g)' = f' + g'$.

- **Multiplicação (Regra do Produto):**

$$(a + b\epsilon)(c + d\epsilon) = ac + ade + bce + bd\epsilon^2 = ac + (bc + ad)\epsilon \quad (8)$$

A componente dual $(bc + ad)$ mapeia exatamente a regra do produto do cálculo diferencial: $(uv)' = u'v + uv'$.

- **Divisão (Regra do Quociente):**

$$\frac{a + b\epsilon}{c + d\epsilon} = \frac{a}{c} + \left(\frac{bc - ad}{c^2} \right) \epsilon \quad (9)$$

- **Visualização da Regra da Cadeia (Composição de Funções):** Seja uma função composta $h(x) = f(g(x))$. Ao aplicar o número dual $x = a + \epsilon$, a avaliação da função interna produz:

$$u = g(a + \epsilon) = g(a) + g'(a)\epsilon \quad (10)$$

Aplicando a função externa f sobre o resultado u , e utilizando a expansão de Taylor de f ao redor de $g(a)$:

$$f(u) = f(g(a) + g'(a)\epsilon) = f(g(a)) + f'(g(a)) \cdot g'(a)\epsilon \quad (11)$$

A componente dual resultante contém precisamente o produto das derivadas locais $f'(g(a)) \cdot g'(a)$, demonstrando que a **Regra da Cadeia** é satisfeita automaticamente a cada passo da execução.

2.1.4 Implementação Computacional em C++

A estrutura `Dual` foi implementada em C++ por meio de uma classe composta por dois atributos do tipo `double`: `val`, que representa o valor primal da expressão, e `der`, que armazena sua derivada. A propagação automática das derivadas é realizada por sobrecarga de operadores e por funções matemáticas que recebem objetos do tipo `Dual`.

Foram implementadas as seguintes operações:

- **Operações aritméticas entre duais:** adição, subtração, multiplicação, divisão e negação unária. Essas operações propagam as derivadas conforme as regras usuais do cálculo diferencial.

- **Operações entre duais e escalares:** adição, subtração, multiplicação e divisão entre um objeto `Dual` e valores do tipo `double`, em ambas as ordens quando aplicável. Por exemplo, a expressão c/x utiliza a derivada

$$\frac{d}{dx} \left(\frac{c}{x} \right) = -\frac{c}{x^2}.$$

- **Funções exponencial e logarítmicas:** foram implementadas as funções $\exp(x)$, $\ln(x)$ e $\log_b(x)$. Para o logaritmo em uma base arbitrária, são impostas as condições $x > 0$, $b > 0$ e $b \neq 1$.
- **Potenciação:** foram implementadas as formas a^x e x^a , em que uma das partes é um número dual. Suas derivadas são dadas, respectivamente, por

$$\frac{d}{dx}(a^x) = a^x \ln(a)$$

e

$$\frac{d}{dx}(x^a) = ax^{a-1}.$$

- **Raiz quadrada:** a função $\sqrt{x}$ é calculada para $x > 0$, utilizando

$$\frac{d}{dx} \sqrt{x} = \frac{1}{2\sqrt{x}}.$$

Valores negativos e o ponto $x = 0$, no qual a derivada não é finita, são tratados como inválidos.

- **Funções trigonométricas:** foram implementadas $\sin(x)$, $\cos(x)$, $\tan(x)$, $\sec(x)$, $\csc(x)$ e $\cot(x)$. Como exemplo,

$$\frac{d}{dx} \tan(x) = \sec^2(x) \quad \text{e} \quad \frac{d}{dx} \cot(x) = -\csc^2(x).$$

As funções com assíntotas verificam valores de seno ou cosseno muito próximos de zero.

- **Funções trigonométricas inversas:** foram implementadas $\arcsin(x)$, $\arccos(x)$ e $\arctan(x)$. Para $\arcsin(x)$ e $\arccos(x)$, o domínio considerado é $-1 < x < 1$, pois suas derivadas não são finitas nos extremos.
- **Tratamento de erros de domínio:** divisões por zero, logaritmos fora do domínio, raízes de valores negativos e avaliações próximas a assíntotas lançam exceções do tipo `std::domain_error`. Dessa forma, os métodos numéricos podem interromper a execução de maneira controlada ao encontrar uma avaliação inválida.

Assim, uma expressão composta, como $f(x) = \sin(x^2) + \ln(x)$, pode ser avaliada diretamente com objetos do tipo `Dual`. A parte primal contém o valor de $f(x)$, enquanto a parte dual contém automaticamente a derivada $f'(x)$, respeitando a regra da cadeia.

3 Implementação dos Métodos

3.0.1 Estruturas de Controle e Registro dos Resultados

O arquivo `types.hpp` centraliza as estruturas e enumerações utilizadas pelos métodos numéricos. Seu objetivo é padronizar a comunicação entre os *solvers*, a interface de relatórios e os testes, permitindo que todos os métodos retornem informações no mesmo formato.

A enumeração `StatusCode` descreve o estado final da execução. Entre os estados possíveis estão:

- **SUCCESS**: uma raiz foi encontrada de acordo com o critério de tolerância;
- **MAX_ITERATIONS_REACHED**: o número máximo de iterações foi atingido sem confirmação de convergência;
- **DOMAIN_INVALID**: ocorreu uma avaliação fora do domínio matemático da função;
- **DIVISION_BY_ZERO**: foi detectada uma divisão por zero, como uma derivada nula no método de Newton-Raphson;
- **DIVERGENCE**: a execução produziu valores não finitos, como *NaN* ou infinito;
- **STAGNATION**: o método deixou de apresentar progresso numérico representável;
- **OTHERS**: representa situações não classificadas nas categorias anteriores.

A enumeração `ConvergenceType` identifica o critério que comprovou a convergência:

- **BY_Y**: o resíduo da função atingiu a tolerância, isto é,

$$|f(x_k)| \leq \text{tol};$$

- **BY_X**: a estimativa de erro na aproximação da raiz atingiu a tolerância;
- **NONE**: não houve convergência confirmada.

O resultado de cada método é armazenado na estrutura `SolverResult`. Seus principais componentes são:

- **status**: código que informa se o método convergiu ou qual falha ocorreu;

- **root**: melhor aproximação de raiz obtida durante a execução;
- **iterations_used**: quantidade de iterações realizadas;
- **convergence**: critério que justificou o encerramento com sucesso;
- **method_name**: nome do método utilizado;
- **history**: vetor contendo os registros detalhados de cada iteração.

Cada elemento do vetor **history** é uma estrutura **IterationData**, composta pelos campos **it**, **x_k**, **f_x**, **error_x**, **error_y** e **step_x**. Esse histórico permite analisar o comportamento do método ao longo das iterações e gerar relatórios comparativos.

O campo **error_y** corresponde ao resíduo:

$$\text{error_y} = |f(x_k)|.$$

Esse valor mede o quão próximo o ponto x_k está de satisfazer a equação $f(x) = 0$. Portanto, é o erro utilizado para a convergência do tipo **BY_Y**.

O campo **error_x** representa uma estimativa de incerteza no eixo x , cuja interpretação depende do método empregado. Na bisseção, ele corresponde à metade da largura do intervalo atual:

$$\text{error_x} = \frac{|b - a|}{2}.$$

Já nos métodos de Newton-Raphson, secante e falsa posição, ele é associado ao módulo da variação entre aproximações consecutivas.

Por fim, **step_x** registra o deslocamento entre duas aproximações consecutivas:

$$\text{step_x} = |x_k - x_{k-1}|.$$

Esse valor é útil para identificar se o método continua produzindo alterações significativas na estimativa da raiz. Um passo pequeno, entretanto, não garante por si só que uma raiz foi encontrada; por isso, o projeto prioriza a verificação do resíduo $|f(x_k)|$. Caso ocorram passos numericamente insignificantes em iterações consecutivas sem que o resíduo atinja a tolerância, o método pode ser classificado como estagnado.

3.1 Bisseção

A implementação do método da bisseção foi estruturada de modo a calcular, antes do início do laço principal, uma estimativa do número de iterações necessário para atingir a tolerância desejada no eixo x . Considerando que a largura do intervalo é reduzida pela metade a cada iteração, o número necessário pode ser estimado por:

$$n = \left\lceil \frac{\log \left(\frac{|b-a|}{\text{tol}} \right)}{\log(2)} \right\rceil. \quad (12)$$

Essa estratégia permite garantir teoricamente a convergência por intervalo, desde que o número calculado não ultrapasse o limite máximo de iterações adotado pelo programa. Assim, ao final das iterações previstas, a estimativa de erro em x é inferior ou igual à tolerância especificada. Como medida de segurança, a implementação também impõe um limite máximo de 100 iterações.

Antes do processo iterativo, são validadas a tolerância e as extremidades do intervalo. Valores não finitos nos parâmetros de entrada, tais como *NaN* ou infinito, tornam a execução inválida. As avaliações da função também são verificadas: caso a função lance uma exceção de domínio, como em um logaritmo de valor não positivo ou em uma divisão por zero, o método é interrompido com o status correspondente. Da mesma forma, caso uma avaliação produza *NaN* ou infinito, o resultado é classificado como divergência.

A verificação da mudança de sinal nos extremos é realizada por meio de `std::signbit`, em vez de utilizar diretamente o produto $f(a)f(b)$. Essa decisão evita possíveis problemas de *overflow* quando os valores da função possuem módulo muito elevado. Caso um dos extremos já apresente valor de função nulo, ele é retornado imediatamente como raiz.

Embora o número de iterações previamente calculado seja suficiente para assegurar a convergência por x , foram mantidas verificações dentro do laço iterativo. Em cada ponto médio calculado, o programa verifica inicialmente a convergência pelo resíduo:

$$|f(x_k)| \leq \text{tol}. \quad (13)$$

Quando essa condição é satisfeita, o método é encerrado com convergência do tipo `BY_Y`. Essa verificação pode permitir o encerramento antecipado da execução, pois uma raiz pode ser encontrada antes de a largura do intervalo atingir a tolerância.

Também é avaliado o erro associado ao intervalo:

$$\text{error_x} = \frac{|b - a|}{2}. \quad (14)$$

Caso esse valor atinja a tolerância, o método termina com convergência do tipo `BY_X`. Portanto, o relatório diferencia claramente a convergência obtida pelo resíduo da função daquela garantida pela redução do intervalo.

Por fim, foi considerado o limite de precisão do tipo `double`. Em determinados casos, especialmente para valores de grande magnitude ou intervalos muito pequenos, o ponto médio calculado pode coincidir com uma das extremidades, isto é, `meio == a` ou `meio == b`. Nessa situação, não existe outro número representável em ponto flutuante entre os extremos, impossibilitando o refinamento do intervalo. A implementação detecta

essa condição para evitar um laço sem progresso numérico. Se a incerteza em x já satisfizer a tolerância, o resultado pode ser aceito por `BY_X`; caso contrário, a execução deve ser interrompida por estagnação numérica.

3.2 Falsa Posição

A implementação da falsa posição recebe um intervalo inicial, uma tolerância e um número máximo de iterações. Inicialmente, são verificadas a validade da tolerância, a finitude dos extremos e a existência de uma mudança de sinal no intervalo. Assim como na bisseção, a comparação de sinais é realizada com `std::signbit`, evitando multiplicações que poderiam causar *overflow*.

As avaliações iniciais da função são protegidas contra exceções de domínio e contra resultados não finitos. Caso um dos extremos já possua resíduo menor ou igual à tolerância, o método encerra imediatamente com convergência do tipo `BY_Y`.

Em cada iteração, o denominador da expressão de interpolação é verificado antes do cálculo da nova aproximação. Caso seja nulo ou numericamente muito pequeno, a execução é interrompida com o status `DIVISION_BY_ZERO`, evitando resultados instáveis. A nova aproximação também é validada com `std::isfinite`, e a função é avaliada novamente dentro de um bloco de tratamento de exceções.

O histórico registra o valor calculado, o resíduo, o passo entre aproximações e uma estimativa de erro em x . Na primeira iteração, essa estimativa corresponde à largura inicial do intervalo; nas seguintes, utiliza-se a distância em relação à aproximação anterior. A convergência é confirmada pelo resíduo:

$$|f(x_k)| \leq \text{tol}. \quad (15)$$

Também foram implementados mecanismos de estagnação. Se a interpolação produzir novamente um extremo do intervalo ou repetir a aproximação anterior sem atingir a tolerância, o método é encerrado com o status `STAGNATION`. Além disso, passos numericamente muito pequenos em iterações consecutivas são detectados utilizando uma tolerância baseada na precisão do tipo `double`.

3.3 Newton-Raphson

A implementação de Newton-Raphson utiliza a classe `Dual` para calcular simultaneamente o valor da função e sua derivada. O chute inicial é avaliado como um número dual cuja componente derivada é igual a 1, permitindo que a derivada seja propagada automaticamente pelas operações da função.

Antes das iterações, são verificadas a validade da tolerância, a finitude do chute inicial e a validade da primeira avaliação da função. Se o resíduo inicial já satisfizer a tolerância,

o método retorna sucesso sem realizar iterações. Caso contrário, o chute inicial é registrado no histórico como iteração zero.

Em cada iteração, são validados o valor da função, sua derivada, o passo calculado e a nova aproximação. Uma derivada nula impede o cálculo da correção de Newton e resulta no status `DIVISION_BY_ZERO`. Valores *NaN* ou infinitos, tanto na avaliação da função quanto na nova aproximação, resultam em `DIVERGENCE`. Exceções de domínio geradas pela função retornam `DOMAIN_INVALID`.

O histórico registra a nova aproximação, o valor da função, o resíduo e o tamanho da correção aplicada:

$$\text{step_x} = |x_{k+1} - x_k|. \quad (16)$$

Nesse método, `error_x` e `step_x` possuem o mesmo valor. Entretanto, um passo pequeno não é utilizado isoladamente como prova de convergência. A confirmação de sucesso ocorre apenas quando o resíduo satisfaz a tolerância, caracterizando convergência do tipo `BY_Y`.

Por fim, a implementação detecta estagnação quando a nova aproximação é idêntica à anterior ou quando passos próximos da resolução numérica do tipo `double` ocorrem em iterações consecutivas sem que o resíduo se torne suficientemente pequeno.

3.4 Secante

A implementação do método da secante utiliza dois chutes iniciais, sem depender da derivada analítica ou automática da função. Inicialmente, são verificadas a tolerância, a finitude dos chutes e os valores da função nos dois pontos. Caso um dos pontos iniciais já apresente resíduo dentro da tolerância, o método retorna sucesso imediatamente por `BY_Y`.

Durante cada iteração, o denominador associado à diferença entre os valores da função é verificado. Se esse denominador for igual a zero, o método não pode prosseguir e retorna `DIVISION_BY_ZERO`. A nova aproximação e a respectiva avaliação da função também são verificadas contra valores não finitos e erros de domínio.

A cada nova aproximação, é inserido um registro no histórico contendo o valor de x_k , o valor de $f(x_k)$, o resíduo e o deslocamento entre as duas últimas aproximações:

$$\text{step_x} = |x_{k+1} - x_k|. \quad (17)$$

Para a secante, esse deslocamento também é utilizado como `error_x` para fins de relatório e de estimativa da ordem de convergência. A convergência efetiva, porém, é confirmada somente pelo resíduo da função, isto é, quando $|f(x_k)| \leq \text{tol}$.

Assim como nos demais métodos iterativos, a implementação inclui tratamento de estagnação. Caso a nova aproximação seja igual à anterior, ou caso passos numericamente

insignificantes se repitam por várias iterações sem reduzir adequadamente o resíduo, o método é encerrado com o status **STAGNATION**.

3.5 Geração de Relatórios

O arquivo `reporter.cpp` concentra as funções responsáveis por apresentar os resultados dos métodos numéricos de forma organizada no terminal. Essas funções utilizam o objeto `SolverResult` e o vetor `history` para exibir tanto o resultado final quanto o comportamento das iterações.

A função `print_history` imprime uma tabela contendo o número da iteração, a aproximação x_k , o valor $f(x_k)$, os erros em x e y e o passo `step_x`. O parâmetro `every` permite controlar a frequência de exibição das iterações, mantendo sempre visíveis a primeira e a última linha do histórico.

As sobrecargas de `report_method` exibem o relatório detalhado de um único resultado. São mostrados o nome do método, o status final, o número de iterações, o tipo de convergência, a raiz aproximada e, quando disponível, a ordem estimada de convergência. Em execuções bem-sucedidas, também são apresentados os erros finais em x e em y .

As funções `report_bissecao`, `report_falsa_posicao`, `report_newton` e `report_secante` executam diretamente o método correspondente e imprimem seu relatório. Há ainda uma versão de `report_method` que recebe um valor do enum `Method`, permitindo selecionar o método sem utilizar textos ou chamadas específicas.

A função `report_all` executa os quatro métodos para uma mesma função e apresenta uma tabela comparativa contendo o número de iterações, o status, o tipo de convergência e a ordem estimada de cada método. Em seguida, são exibidos os erros finais dos métodos que convergiram.

Por fim, `benchmark_metodos` produz um ranking dos métodos com base na ordem estimada de convergência. Os resultados com ordem válida são priorizados, seguidos pelos métodos que convergiram sem uma estimativa disponível. A função `run_suite_comparison` permite executar essa comparação para diversos casos de teste, facilitando a análise de diferentes funções, intervalos e tolerâncias.